`42678` from the photograph, which can then be associated with the date the photo was taken and it's location, and stored in a database.

Listing 2.2 Example code to extract a meter reading from an image of an analog meter, and digitize it using OCR.

```
import cv2, pytesseract
import numpy as np
from PIL import Image, ImageOps

def extract_meter_reading(image_path, display_results=True):
  img = Image.open(image_path)
  crop_box = (1000, 890, 1700, 1030) #1
  cropped_img = img.crop(crop_box) #2
  gray_img = ImageOps.grayscale(cropped_img) #3
  inverted_img = ImageOps.invert(gray_img) #4
  meter_reading = pytesseract.image_to_string( #5
    inverted_img,
    config='--psm 7 -c tessedit_char_whitelist=0123456789'
  )
  meter_reading = meter_reading.strip() #6

  return meter_reading
```

#1 This is the bounding box of the digits in the image.
#2 Crop the image to the digit bounding box.
#3 Convert the cropped image to grayscale.
#4 Invert the grayscale image. This often helps OCR for dark digits on a light background (like the meter reading).
#5 Use OCR to extract text. Treat the image as a single line of text (often good for meter readings) and restrict OCR to only digits.
#6 Clean up the extracted text (remove whitespace, newlines).

You may wonder if it is worth going to the trouble of taking a photograph of the meter daily and running the image through this process to extract the reading. Why not simply enter the reading into a computer to digitize it? By placing an IoT camera over the analog meter, I can use OCR on the photographs it takes to fully automate taking a daily meter reading.

2.3.3 Videos

Static capture methods provide snapshots, but video streams introduce continuous temporal monitoring, updating the digital representation of your